

BlockQuiz
Web-Based Block Programming Platform with Auto-Grading for
students

Fabio Plunser



B.Sc. Thesis

Supervisor: Ass. Prof. Dr. Michael Vierhauser
19th March 2026

Abstract

This thesis presents the design and implementation of BlockQuiz, a web-based block programming learning platform with automated grading capabilities, specifically designed for children aged 8-12 years. As computational thinking becomes increasingly integrated into educational curricula, notably through Austria’s mandatory “Digitale Grundbildung” subject introduced in 2022 Bundesministerium für Bildung, Wissenschaft und Forschung (2022), there is a growing need for accessible, teacher-friendly programming tools that provide immediate feedback without requiring extensive technical expertise from educators.

Unlike existing platforms such as Scratch, Tynker, or Microsoft MakeCode, which tend toward open-ended creative projects or hardware-specific applications, BlockQuiz focuses on short, tightly constrained programming puzzles with clear learning objectives. The platform implements two primary exercise types: Input/Output puzzles for algorithmic thinking and 2D Graphics exercises for visual reasoning and geometry concepts. Each exercise features a constrained block toolbox, reducing cognitive load while guiding learners toward solutions.

The technical implementation leverages modern web technologies including SvelteKit, Google Blockly for the visual programming interface, and a privacy-first architecture enabling self-hosted deployment via Docker. Key contributions include a secure client-side code execution sandbox with timeout protection, a flexible auto-grading system supporting multiple validation strategies, and a teacher-oriented content management system for exercise authoring without programming knowledge.

The platform supports internationalization (German and English), implements progressive hint systems, and captures learning analytics for educational research. An evaluation with target users demonstrates the platform’s usability and effectiveness in teaching core programming concepts including sequences, loops, conditionals, and simple functions.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Ass. Prof. Dr. Michael Vierhauser, for his invaluable guidance, constructive feedback, and continuous support throughout this project. His expertise in software quality and educational technology significantly shaped the direction of this work.

I am also grateful to the Quality Engineering research group at the University of Innsbruck for providing an inspiring academic environment and the resources necessary to complete this thesis.

Special thanks go to all participants who took part in the usability evaluation, providing essential feedback that helped improve the platform's design and functionality.

Finally, I would like to thank my family and friends for their encouragement and patience during the development of this project.

Contents

Abstract	i
Acknowledgments	iii
Contents	v
List of Figures	vii
List of Tables	ix
Declaration	xi
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Goals and Contributions	1
1.3 Target Audience	2
1.4 Scope and Limitations	2
1.5 Document Structure	2
2 Background and Related Work	5
2.1 Block-Based Programming Environments	5
2.1.1 Scratch	5
2.1.2 Microsoft MakeCode	5
2.1.3 Google Blockly	6
2.1.4 Code.org	6
2.1.5 Comparison Summary	6
2.2 Automated Assessment in Programming Education	6
2.2.1 Test-Based Grading	7
2.2.2 State-Based Grading	7
2.2.3 Benefits of Immediate Feedback	7
2.3 Learning Theory Foundations	7
2.3.1 Constructionism	8
2.3.2 Cognitive Load Theory	8
2.3.3 Age-Appropriate Design	8
2.3.4 Privacy and Data Protection	9
2.3.5 Technical foundations	10

3	Requirements Analysis	13
3.1	Overview	13
3.2	Stakeholders	13
3.3	Functional Requirements	13
3.4	Non-Functional Requirements	15
3.5	Use Cases	16
4	System Design	17
4.1	Design decisions	17
4.1.1	Block Editor: Google Blockly	17
4.1.2	Code Execution: Client-Side Sandbox	17
4.1.3	Database: SQLite with Drizzle ORM	18
4.2	Architectural Overview	18
4.2.1	Component Descriptions	18
4.2.2	Exercise Type Framework	19
4.3	Data Model	20
4.3.1	Core Tables	20
4.3.2	Relationships	20
4.4	Security Model	20
4.4.1	Threat Model	20
4.4.2	Mitigations	20
5	Implementation	23
5.1	Development Setup	23
5.2	Blockly Integration with Svelte 5	23
5.3	Canvas2D Rendering Engine	23
5.4	Grading Pipeline	23
5.5	Teacher Interface and Course Management	23
5.6	Authentication and Authorization	23
6	Evaluation	25
6.1	Evaluation Methodology	25
6.2	Heuristic Evaluation	25
6.3	Grader Accuracy	25
6.4	Limitations	25
7	Conclusion	27
7.1	Summary of Contributions	27
7.2	Limitations	27
7.3	Future Work	27
	Bibliography	29

List of Figures

4.1	System Architecture Overview	18
-----	--	----

List of Tables

2.1	Comparison of Block-Based Programming Platforms	6
4.1	Block Editor Selection	17
4.2	Code Execution Options	17
4.3	Database Selection	18

Declaration

By my own signature I declare that I produced this work as the sole author, working independently, and that I did not use any sources and aids other than those referenced in the text. All passages borrowed from external sources, verbatim or by content, are explicitly identified as such.

Signed: Date:

1 Introduction

Computational thinking has emerged as a fundamental skill for the 21st century, increasingly recognized as essential for all learners regardless of their future career paths Wing (2006); Grover and Pea (2013). In Austria, this recognition materialized with the introduction of “Digitale Grundbildung” (Digital Basic Education) Bundesministerium für Bildung, Wissenschaft und Forschung (2022) as a mandatory subject for secondary education in 2022/23, reflecting a broader European trend toward integrating computational concepts into primary and secondary curricula.

1.1 Motivation and Problem Statement

While numerous block-based programming environments exist, including Scratch, Tynker, Microsoft MakeCode, and Snap!, these platforms present several challenges when deployed in classroom settings with younger learners:

- **Open-ended complexity:** Platforms like Scratch emphasize creative, open-ended projects that can overwhelm beginners and make assessment difficult.
- **Hardware dependencies:** Tools such as MakeCode are often tied to specific hardware (micro:bit, Arduino), limiting accessibility.
- **Lack of automated feedback:** Most existing solutions require manual teacher intervention for assessment, creating bottlenecks in larger classrooms.
- **Privacy concerns:** Cloud-based platforms raise data protection issues, particularly with younger users in European educational contexts.

Teachers, especially those without programming backgrounds, need tools that allow them to create and manage exercises without technical expertise while providing students with immediate, meaningful feedback on their solutions.

1.2 Goals and Contributions

This thesis addresses these challenges through the design and implementation of BlockQuiz, a web-based block programming learning platform. The primary contributions are:

1. **Constrained exercise model:** A framework for creating focused programming puzzles with per-exercise block toolboxes, reducing cognitive load and enabling clear learning objectives.

2. **Client-side auto-grading:** A secure sandbox execution environment with multiple grading strategies for I/O and graphical (Turtle) exercises, providing immediate feedback without server-side code execution.
3. **Teacher-oriented authoring:** A content management system enabling non-programmers to create, edit, and organize exercises and courses.
4. **Privacy-first architecture:** A self-hostable solution deployable via Docker in school networks, with optional guest mode requiring no personal data.
5. **Internationalization:** Full support for German and English, with extensible localization infrastructure.

1.3 Target Audience

The platform primarily targets children aged 8–12 years, a developmental stage where learners can handle block-based logic, loops, and conditionals while benefiting from visual, immediate feedback. Secondary users include teachers who author and manage exercises, and administrators who deploy and maintain the platform.

1.4 Scope and Limitations

The minimum viable product (MVP) encompasses:

- Two exercise types: I/O puzzles and Turtle Graphics
- Ten curated exercises covering sequences, loops, conditionals, and simple functions
- A minimal CMS for exercise and course management
- Client-side sandboxed execution with configurable timeouts
- German and English localization
- Docker-based deployment

Explicitly out of scope are: full classroom management with detailed analytics dashboards, complex simulators beyond 2D canvas/turtle, and integration with institutional single sign-on systems (SAML/SSO), though the architecture supports future extensions.

1.5 Document Structure

The remainder of this thesis is organized as follows:

- **Chapter 2: Background and Related Work** reviews existing block-based programming platforms, educational research on computational thinking, and technical foundations including Google Blockly.
- **Chapter 3: Requirements Analysis** details functional and non-functional requirements derived from educational and technical constraints.
- **Chapter 4: System Design** presents the architecture, including the exercise model, grading framework, and CMS design.

- **Chapter 5: Implementation** describes the technical realization using SvelteKit, Blockly, and the sandbox execution environment.
- **Chapter 6: Evaluation** reports on usability testing with target users and discusses results.
- **Chapter 7: Conclusion** summarizes contributions and outlines future work.

2 Background and Related Work

2.1 Block-Based Programming Environments

Block-based programming environments have emerged as a dominant approach for introducing computational concepts to novice programmers, particularly children. By representing code as interlocking visual blocks, these environments eliminate syntax errors and reduce the cognitive load associated with textual programming languages Weintrop and Wilensky (2015); Maloney et al. (2010).

Empirical studies of Scratch-based learning further suggest that block-based programming can support the development of programming concepts, problem solving, and curricular integration when tasks are appropriately scaffolded Meerbaum-Salant et al. (2013); Kalelioğlu and Gülbahar (2014); Sáez-López et al. (2016).

2.1.1 Scratch

Scratch, developed at the MIT Media Lab, is perhaps the most widely recognized block-based programming environment for children Resnick et al. (2009). Launched in 2007, Scratch emphasizes creative expression through the creation of interactive stories, games and animations. The platform has been very successful, with millions of projects shared in its online community.

However, Scratch’s open-ended nature presents challenges for structured classroom instruction:

- Projects can become arbitrarily complex, making assessment difficult
- No built-in auto-grading or immediate feedback mechanism
- The large block library can overwhelm beginners
- Cloud-based storage raises privacy concerns for European users

2.1.2 Microsoft MakeCode

Microsoft MakeCode extends block-based programming to physical computing, supporting hardware platforms such as the *micro:bit*, Arduino, and Adafruit Circuit Playground Ball et al. (2019); Devine et al. (2019); Austin et al. (2020). MakeCode provides a polished development experience with integrated simulation, block-to-text transitions, and hardware deployment.

While MakeCode excels at hardware integration, its complexity and hardware focus limit its applicability for pure algorithmic thinking exercises. Classroom studies around the *micro:bit* ecosystem emphasize creative and physical-computing use cases rather than tightly constrained worksheet-style tasks Sentance et al. (2017a,b); Bachinger and Teufel (2022). The platform is designed for project-based learning rather than structured, gradeable exercises.

2.1.3 Google Blockly

Google Blockly is not a complete programming environment but rather a library for building block-based programming interfaces Google (2024). MakeCode for example uses it under the hood. Blockly provides:

- A robust block rendering and connection engine
- Extensible code generators for multiple target languages
- Internationalization support with block label translations
- Accessibility features including keyboard navigation
- Customizable toolboxes and block categories

Blockly’s library nature makes it ideal for building custom educational platforms where the exercise structure, grading logic and user interface can be tailored to specific needs. BlockQuiz leverages Blockly as its foundation, building a complete learning platform around the library.

2.1.4 Code.org

Code.org offers curriculum-aligned computer science courses with structured exercises and automatic grading. The platform has been widely adopted in the United States and provides a good scaffold for learners. However, Code.org’s closed ecosystem nature and US-centric curriculum limits its applicability for Austrian schools, and its cloud-only deployment model raises data protection concerns.

2.1.5 Comparison Summary

Table 2.1 summarizes the key characteristics of existing platforms and how BlockQuiz differentiates itself.

Table 2.1: Comparison of Block-Based Programming Platforms

Platform	Auto-grading	Teacher CMS	Self-hostable	Open Source
Scratch	No	No	No	Yes
MakeCode	Partial	No	No	Yes
Code.org	Yes	Limited	No	Partial
Blockly	N/A (library)	N/A	Yes	Yes
BlockQuiz	Yes	Yes	Yes	Yes

2.2 Automated Assessment in Programming Education

Automated assessment of programming exercises has been an active research area since the 1960s, with early classroom systems already described by Hollingsworth, Naur, and Forsythe and Wirth Hollingsworth (1960); Naur (1964); Forsythe and Wirth (1965); later surveys document the continued development of this research area Ala-Mutka (2005); Douce et al. (2005); Ihantola et al. (2010); Messer et al. (2024). Modern auto-graders employ various strategies to evaluate student code, each with distinct advantages and limitations.

2.2.1 Test-Based Grading

The most common approach executes student code against a set of test cases Douce et al. (2005); Ala-Mutka (2005), comparing actual output to expected output. This approach is well suited for Input/Output (I/O) exercises, where correctness can be defined in terms of input-output pairs.

Key considerations for test-based grading include:

- **Visible vs. hidden tests:** Visible tests provide immediate feedback and learning scaffolds, while hidden tests prevent hardcoding of solutions.
- **Output normalization:** Handling whitespace, case sensitivity, and formatting differences.
- **Partial credit:** Awarding points for partially correct solutions.
- **Feedback quality:** Providing meaningful error messages rather than simple pass/fail indicators.

2.2.2 State-Based Grading

For visual programming exercises such as turtle graphics, state-based grading evaluates the final state of the execution environment rather than textual output. This includes:

- Position and orientation of graphical elements
- Command sequences executed during the program
- Canvas or grid state at program termination

State-based grading allows for more flexible solutions, as multiple command sequences can produce equivalent final states Malmi et al. (2004). BlockQuiz implements both exact command matching and final state comparison with configurable tolerance thresholds.

2.2.3 Benefits of Immediate Feedback

Research on formative assessment consistently demonstrates the value of immediate feedback for learning Black and Wiliam (1998); Hattie and Timperley (2007); Shute (2008). Auto-grading enables instant feedback that would be impossible to provide manually in a classroom setting, allowing students to:

- Iterate quickly on solutions
- Identify and correct misunderstandings immediately
- Progress at their own pace without waiting for teacher review

2.3 Learning Theory Foundations

The design of BlockQuiz is informed by learning theories that guide decisions in exercise design, feedback mechanisms and scaffolding approaches.

2.3.1 Constructionism

Seymour Papert’s constructionism builds on Piaget’s constructivism, arguing that learning is particularly effective when learners are actively constructing artifacts that are personally meaningful (Papert (1980); Papert and Harel (1991)). Papert’s work with Logo and turtle graphics demonstrated how programming could serve as a medium for exploring mathematical concepts through construction.

BlockQuiz embraces constructionist principles by:

- Providing immediate visual feedback through turtle graphics (as in Papert (1980))
- Allowing learners to experiment and observe consequences
- Making abstract concepts (angles, distances, repetition) tangible

2.3.2 Cognitive Load Theory

Cognitive Load Theory (Sweller (1988)) distinguishes between intrinsic load (inherent to the material), extraneous load (caused by poor instructional design) and germane load (contributing to learning). Block-based programming reduces extraneous cognitive load by:

- Eliminating syntax errors and the need to memorize keywords
- Providing visual scaffolding through block shapes that indicate valid connections
- Constraining the problem space through per-exercise toolboxes

BlockQuiz’s constrained exercise model directly applies cognitive load principles by limiting available blocks to those relevant for each learning objective, reducing the extraneous load of navigating a large block library.

Zone of Proximal Development

Vygotsky’s Zone of Proximal Development (ZPD) (Vygotsky (1980); Shabani et al. (2010)) describes the gap between what a learner can do independently and what they can achieve with guidance. Effective instruction targets this zone, providing appropriate scaffolding that can be gradually removed.

BlockQuiz implements scaffolding through:

- Progressive hint systems that reveal guidance on demand
- Starter blocks that provide partial solutions for complex exercises
- Difficulty progression within courses that builds on prior knowledge

2.3.3 Age-Appropriate Design

Design for children aged 8-12 requires understanding developmental psychology and applying principles from child-computer interaction research.

Developmental Considerations

In classical Piagetian terms, many children in roughly the 7–11/12 age range are in the concrete operational stage Piaget and Inhelder (1969). This stage is commonly characterized by:

- Ability to perform logical operations on concrete objects
- Development of conservation and reversibility concepts
- More limited abstract reasoning than in later formal-operational thinking

This has implications for exercise design:

- Younger children (8-9) often benefit from concrete, visual exercises like turtle graphics
- Abstract concepts (variables, conditionals) should be introduced with concrete examples
- Older children (11-12) may increasingly handle more abstract I/O-style exercises when they are appropriately scaffolded

Interface Design for Children

Research on child-computer interaction suggests several design principles Read and Bekker (2008):

- **Large targets:** Interactive elements should be large enough for less precise motor control
- **Minimal text:** Instructions should be concise and use simple vocabulary
- **Visual feedback:** Immediate, visual responses to actions
- **Forgiving interfaces:** Undo functionality and tolerance for errors
- **Playful aesthetics:** Engaging visual design appropriate for the age group

BlockQuiz applies these principles through large block targets, concise exercise descriptions, immediate visual feedback from the canvas, and encouraging feedback messages.

2.3.4 Privacy and Data Protection

The General Data Protection Regulation (European Union (2016)) imposes strict requirements on processing children’s data within the European Union. These requirements are particularly relevant for educational software deployed in schools.

GDPR Requirements for Children’s Data

Key GDPR provisions affecting BlockQuiz include:

- **Article 8:** Parental consent required for children under 16 (member states may lower to 13)
- **Article 5:** Data minimization principle – collect only necessary data
- **Article 25:** Privacy by design and by default
- **Articles 15-17:** Rights to access, rectification and reuse

Privacy-by-Design in BlockQuiz

BlockQuiz implements privacy-by-design principles through:

- **Guest mode:** Default operation without accounts or personal data
- **Local progress storage:** Browser-based persistence (LocalStorage) eliminating server-side data collection
- **Self-hosting:** Docker deployment enables schools to run the platform within their network, avoiding data transfers to external servers
- **No external services:** Assets are self-hosted; no CDNs or analytics services that would transfer data externally

The approach is that BlockQuiz is not a SaaS platform but rather a finished package a school can deploy on their own infrastructure, or even if they want just on an old computer. The data belongs to the school and is not transferred to any third party.

2.3.5 Technical foundations

SvelteKit and Svelte 5

SvelteKit is a modern web application framework built on Svelte, a component framework that compiles components to efficient JavaScript at build time rather than using a virtual DOM at runtime. Svelte 5 introduces "runes" - a powerful reactive primitive system that enables reactive state in regular JavaScript/TypeScript files:

Listing 2.1: Example of a Svelte 5 rune

```
// Reactive state in .svelte.ts files
export class Turtle {
  x = $state(200); // Reactive property
  y = $state(200);
  move(distance: number) {
    this.x += distance * Math.sin(this.angle);
    // UI automatically updates when x changes
  }
}
```

This capability enables clean separation between business logic (exercises engines, graders) and UI components while maintaining automatic reactivity.

SQLite and Drizzle ORM

SQLite provides a serverless, embedded database ideal for single-school deployments. Combined with Drizzle ORM for type-safe database access and migrations, this stack offers:

- Zero configuration database setup
- Portable single-file storage
- Type-safe queries with compile-time verification
- Straightforward backup (file copy)

Containerized Deployment

Docker containerization enables consistent deployment across environments. A single `docker-compose` command starts the complete application, making it accessible even for school IT administrators without extensive DevOps experience.

3 Requirements Analysis

3.1 Overview

This section presents the functional and non-functional requirements derived from educational needs, technical constraints and privacy considerations. Requirements were gathered through analysis of existing platforms, Austrian curriculum documents and established principle of educational software design.

3.2 Stakeholders

The platform serves multiple stakeholder groups with distinct needs:

Studnets (8-12 years) Require engaging, achievable challenges with immediate feedback. Usability and age-appropriate design are critical. **Teachers** Need easy content creation without programming knowledge, progress tracking and minimal setup overhead. **School IT Administrators** Require simple deployment, easy maintenance and compliance with data protection regulations. **Parents** Expect educational value, safety and privacy protection for their children.

3.3 Functional Requirements

Exercise Model (FR-1)

- **FR-1.1:** Each exercise has a unique identifier, title, description, type (io, turtle) and tags/topics
- **FR-1.2:** Each exercise defines a toolbox (whitelist of allowed blocks), an optional starter program and a reference solution
- **FR-1.3:** Each exercise includes progressive hints (minimum two) and grader configuration specifying type tests, normalization rules and tolerances.
- **FR-1.4:** All textual content (hints, feedback) is provided in German and English with fallback to the default language for missing translations

Course Management (FR-2)

- **FR-2.1:** Exercises reside in a central pool, filterable by topic, type, difficulty and age band.
- **FR-2.2:** Courses are ordered sequence of exercises, enabling progressive learning paths.
- **FR-2.3:** Exercises can be reused across multiple courses.

Authoring System (FR-3)

- **FR-3.1:** Authors can create, edit, clone and archive exercises
- **FR-3.2:** A validation linter checks for required fields, complete translations, toolbox consistency and presence of hidden test.
- **FR-3.3:** Preview mode allows authors to test exercises before publishing
- **FR-3.4:** Import/export functionality for exercise backup and sharing as JSON format

Execution and Sandbox (FR-4)

- **FR-4.1:** Blocks generate executable code (JavaScript) via Blockly's code generators.
- **FR-4.2:** Code executes in an isolated sandbox environment (iframe with restricted permissions).
- **FR-4.3:** The sandbox exposes only whitelisted APIs; no access to DOM, network, or browser storage.
- **FR-4.4:** Configurable resource limits include maximum execution time (default 5 seconds) and infinite loop detection via command counting.

Auto-Grading (FR-5)

- **FR-5.1:** I/O exercises support multiple test cases (visible and hidden), output normalization (trim, whitespace collapse, case insensitivity) and detailed per-test feedback.
- **FR-5.2:** Turtle/Graphics exercises support grading via command log comparison or final state evaluation (position, angle) with configurable tolerance thresholds.
- **FR-5.3:** Grading is deterministic through seeded random number generation and prohibition of time-dependent operations
- **FR-5.4:** Results include overall pass/fail status, numerical score and detailed per-test results with expected/actual values.

User Management (FR-6)

- **FR-6.1:** Role-based access control distinguishes Students, Teachers and Administrators
- **FR-6.2:** Guest mode operates without accounts, storing progress locally in the browser
- **FR-6.3:** Progress tracking records status (not started, attempted, passed), best score, attempt count, and hint usage per exercise

Internationalization (FR-7)

- **FR-7.1:** System UI is available in German and English with user-selectable language
- **FR-7.2:** Exercise content supports both languages with automatic fallback and missing-key logging
- **FR-7.3:** Block labels are localized via Blockly's internationalization mechanisms

3.4 Non-Functional Requirements

Security (NFR-1)

- **NFR-1.1:** Sandbox isolation via iframe with sandbox attributes and strict Content Security Policy headers
- **NFR-1.2:** No student code access to DOM, window, network or storage APIs
- **NFR-1.3:** Input validation on all API endpoints using schema validation
- **NFR-1.4:** Role-based authorization enforcing least-privilege access

Privacy (NFR-1)

- **NFR-1.1:** Sandbox isolation via iframe with sandbox attributes and strict Content Security Policy headers
- **NFR-1.2:** No student code access to DOM, window, network or storage APIs
- **NFR-1.3:** Input validation on all API endpoints using schema validation
- **NFR-1.4:** Role-based authorization enforcing least-privilege access

Performance (NFR-1)

- **NFR-1.1:** Sandbox isolation via iframe with sandbox attributes and strict Content Security Policy headers
- **NFR-1.2:** No student code access to DOM, window, network or storage APIs
- **NFR-1.3:** Input validation on all API endpoints using schema validation
- **NFR-1.4:** Role-based authorization enforcing least-privilege access

Accessibility (NFR-1)

- **NFR-1.1:** Sandbox isolation via iframe with sandbox attributes and strict Content Security Policy headers
- **NFR-1.2:** No student code access to DOM, window, network or storage APIs
- **NFR-1.3:** Input validation on all API endpoints using schema validation
- **NFR-1.4:** Role-based authorization enforcing least-privilege access

Maintainability (NFR-1)

- **NFR-1.1:** Sandbox isolation via iframe with sandbox attributes and strict Content Security Policy headers
- **NFR-1.2:** No student code access to DOM, window, network or storage APIs
- **NFR-1.3:** Input validation on all API endpoints using schema validation
- **NFR-1.4:** Role-based authorization enforcing least-privilege access

3.5 Use Cases

4 System Design

4.1 Design decisions

Key architectural decisions were made based on the requirements and constraints identified in the analysis phase.

4.1.1 Block Editor: Ggoogle Blockly

Table 4.1: Block Editor Selection

Option	Pros	Cons	Decision
Google Blockly	Mature, documented, accessible, customizable toolbox	Larger bundle size	✓
Custom implementation	Full control, smaller size	Development time, maintenance burden	
Scratch Blocks	Familiar Scratch aesthetics	Less flexible, tied to Scratch ecosystem	

Rationale: Blockly provides proven reliability, accessibility features and the flexibility to define per-exercise toolboxes. The bundle size trade-off is acceptable given lazy loading optimizations.

4.1.2 Code Execution: Client-Side Sandbox

Table 4.2: Code Execution Options

Option	Pros	Cons	Decision
Client-side (iframe sandbox)	No server load, immediate response	Limited isolation compared to server-side, potential security risks	✓
Server-side (VM/Container)	Better isolation	Added complexity, latency	
WebAssembly sandbox	Best isolation	Significant development effort	

Rationale: For educational code from children, client-side execution with loop traps and API whitelisting provides sufficient security. Server-side execution would add complexity and latency without proportional benefits for this use case.

4.1.3 Database: SQLite with Drizzle ORM

Table 4.3: Database Selection

Option	Pros	Cons	Decision
SQLite	No setup, embedded, portable	Single-writer limitation	✓
PostgreSQL	Scalable, concurrent writes	Requires server, more complex	

Rationale: QLite oers the best balance for single-school deployments. The Drizzle ORM provides type-safe queries and migration management. The schema is designed to be portable, enabling future migration to PostgreSQL if needed.

4.2 Architectural Overview

The system follows a layered architecture separating presentation, business logic and data access concerns.

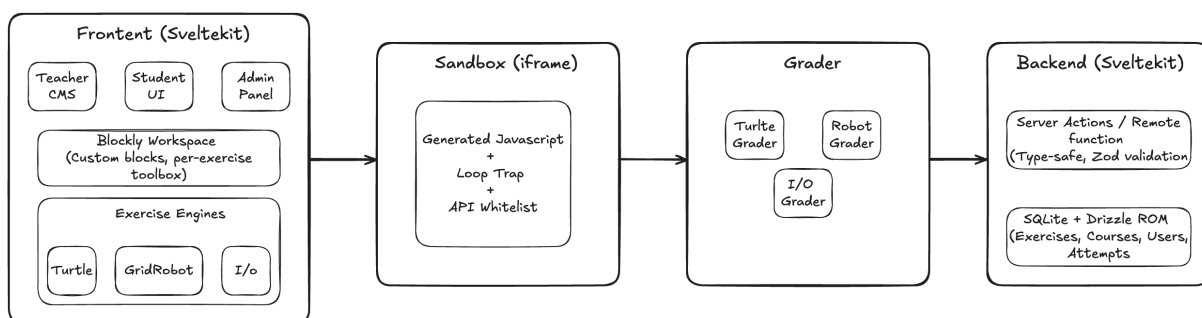


Figure 4.1: System Architecture Overview

4.2.1 Component Descriptions

Frontend Layer

The frontend is built with SvelteKit and comprises three main interfaces:

- **Student Interface:** Displays exercises, the Blockly workspace, canvas visualization and grading feedback. Designed for simplicity and age-appropriateness.
- **Teacher Interface:** Enables exercise creation and editing, course organization and progress monitoring. Includes visual editors for paths and grids.
- **Admin Interface:** Manages users, system configuration and plugin administration.

Exercise Engines

Exercise engines encapsulate the state and behavior for each exercise type:

- **Turtle:** Manages position, angle, pen state, and path history. Commands include *move(distance)*, *turn(degrees)*, and pen control.

- **GridRobot:** Manages grid position, direction and inventory. Command sinclude directional movement, collection and interaction,.
- **I/O:** Manages input consumption and output accumulation for stdin/stdout-style exercises.

Sandbox

The sandbox provides secure code execution through:

- An ifram with *sandbox="allow-scripts"* attribute
- PostMessage-based communication protocol
- Whitelisted API surface exposing only exercise-specific functions
- Loop trap injection detecting infinite loops
- Timeout enforcement terminating long-running executions

Graders

Type-specific grades evaluate student solutions.

- **Turtle Grader:** Compares final state (position, angle) or command sequencees against expected values with configurable tolerances.
- **I/O Grader:** Compares program output against expected output with nromalization options.

4.2.2 Exercise Type Framework

The exercise framework is dsigned for extensibility through a common base class and grading interface.

Abstract Base: Canvas2D

All visual exerices types inherit from Canvas2D, which provides:

- Reactive state for position coordinates
- Command logging for grading
- Reset and comparison methods
- Canvas rendering hooks

Concrete Implementations

Turle Extends Canvas2D with angle, pen state, and move/turn methods. **GridRobot** Extends Canvas@d with discrete grid positioning, direction and inverntory management. **I/O** Implements a non-visual exercise type with input consumption and output generation capabilities.

New exercise types can be added by:

1. Implementing a class extending Canvas2D
2. Defining corresponding Blockly blocks and generators
3. Implementing a grade for the new type

4.3 Data Model

The data model or database schema supports exercises, courses, user progress and audit logging.

4.3.1 Core Tables

- **users** Stores user credentials, roles (student, teacher, author, admin) and preferences.
- **courses** Contains course metadata, ordered exercises list via **courseExercises** junction table, and enrollment via **courseUsers**.
- **exercises** Stores exercise definitions including type (io/turtle/robot), JSON content (localized title/description), JSON config (type-specific grader settings), toolbox XML, starter XML and hints.
- **attempts** Records student attempts with workspace XML, grading results, score and timestamp.
- **auditLogs** Tracks all admin actions like login, account creation, password reset for accountability and auditing.
- **exerciseVersions** Stores snapshots of exercises for versioning support
- **translations** Custom i18n table: locale/keys/value rows used for runtime language lookup

4.3.2 Relationships

- Courses contain ordered lists of exercises (many-to-many with ordering)
- Users have roles determining access to CMS and admin features
- Attempts link users to exercises with temporal tracking

4.4 Security Model

4.4.1 Threat Model

Primary threats addressed by the security model include:

- Malicious student code attempting DOM access or infinite loops
- Cross-site scripting through user-generated content
- Unauthorized access to exercises or administrative functions

4.4.2 Mitigations

- **Iframe Sandbox** The *sandbox="allow-scripts"* attribute prevents navigation, from submission and plugin access.
- **Loop Trap** Command counting detects and terminates infinite loops.
- **API Whitelist** Only exercise-specific functions are exposed; no access to window, document, or network APIs.

- **CSP Headers** Strict Content Security Policy prevents inline scripts and external resource loading.
- **Input Validation** Zod schema validation on all API inputs prevents injection.
- **Role-Based Access** Teachers cannot access admin functions; students cannot access CMS.

5 Implementation

5.1 Development Setup

5.2 Blockly Integration with Svelte 5

5.3 Canvas2D Rendering Engine

5.4 Grading Pipeline

5.5 Teacher Interface and Course Management

5.6 Authentication and Authorization

6 Evaluation

6.1 Evaluation Methodology

6.2 Heuristic Evaluation

6.3 Grader Accuracy

6.4 Limitations

7 Conclusion

7.1 Summary of Contributions

7.2 Limitations

7.3 Future Work

Bibliography

- Kirsti M. Ala-Mutka. A survey of automated assessment approaches for programming assignments. *Computer Science Education*, 15(2):83–102, 2005. doi: 10.1080/08993400500150747.
- Jonny Austin, Howard Baker, Thomas Ball, James Devine, Joe Finney, Peli de Halleux, Steve Hodges, Michal Moskal, and Gareth Stockdale. The bbc micro:bit. *Communications of the ACM*, 63(3):62–69, 2020. doi: 10.1145/3368856.
- Alois Bachinger and Martin Teufel, editors. *Digitale Bildung in der Sekundarstufe: Denken lernen Probleme lösen mit BBC micro:bit V1 + V2*. Austro.Tec, Grieskirchen, 2 edition, 2022. ISBN 978-3-200-08435-3. URL https://microbit.eeducation.at/images/c/c7/Buch_microbit_sek_i-Auflage_2022_20220905_30MB.pdf.
- Thomas Ball, Abhijith Chatra, Peli de Halleux, Steve Hodges, Michal Moskal, and Jacqueline Russell. Microsoft makecode: Embedded programming for education, in blocks and typescript. In *Proceedings of the 2019 ACM SIGPLAN Symposium on SPLASH-E*, pages 7–12, 2019. doi: 10.1145/3358711.3361630.
- Paul Black and Dylan Wiliam. Assessment and classroom learning. *Assessment in Education: Principles, Policy & Practice*, 5(1):7–74, 1998. doi: 10.1080/0969595980050102.
- Bundesministerium für Bildung, Wissenschaft und Forschung. Verordnung des bundesministers für bildung, wissenschaft und forschung, mit der die verordnung über die lehrpläne der mittelschulen sowie die verordnung über die lehrpläne der allgemeinbildenden höheren schulen geändert werden. <https://www.ris.bka.gv.at/eli/bgbl/II/2022/267/20220706>, 2022. BGBl. II Nr. 267/2022.
- James Devine, Joe Finney, Peli de Halleux, Michal Moskal, Thomas Ball, and Steve Hodges. Makecode and codal: Intuitive and efficient embedded systems programming for education. *Journal of Systems Architecture*, 98:468–483, 2019. doi: 10.1016/j.sysarc.2019.05.005.
- Christopher Douce, David Livingstone, and James Orwell. Automatic test-based assessment of programming: A review. *Journal on Educational Resources in Computing*, 5(3):4, 2005. doi: 10.1145/1163405.1163409.
- European Union. Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 (General Data Protection Regulation). <https://eur-lex.europa.eu/eli/reg/2016/679/oj>, 2016. OJ L 119, 4.5.2016, pp. 1–88.
- George E. Forsythe and Niklaus Wirth. Automatic grading programs. *Communications of the ACM*, 8(5):275–278, 1965. doi: 10.1145/364914.364937.
- Google. Blockly: A library for building visual programming editors. <https://developers.google.com/blockly>, 2024. Accessed: 2024-12-01.

- Shuchi Grover and Roy Pea. Computational thinking in k–12: A review of the state of the field. *Educational Researcher*, 42(1):38–43, 2013. doi: 10.3102/0013189X12463051.
- John Hattie and Helen Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007. doi: 10.3102/003465430298487.
- Jack Hollingsworth. Automatic graders for programming classes. *Communications of the ACM*, 3(10):528–529, 1960. doi: 10.1145/367415.367422.
- Petri Ihantola, Tuukka Ahoniemi, Ville Karavirta, and Otto Seppälä. Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, pages 86–93. ACM, 2010. doi: 10.1145/1930464.1930480.
- Filiz Kalelioğlu and Yasemin Gülbahar. The effects of teaching programming via scratch on problem solving skills: A discussion from learners’ perspective. *Informatics in Education*, 13(1):33–50, 2014. doi: 10.15388/infedu.2014.03.
- Lauri Malmi, Ville Karavirta, Ari Korhonen, Jussi Nikander, Otto Seppälä, and Panu Silvasti. Visual algorithm simulation exercise system with automatic assessment: Trakla2. *Informatics in Education*, 3(2):267–288, 2004. doi: 10.15388/infedu.2004.19.
- John Maloney, Mitchel Resnick, Natalie Rusk, Brian Silverman, and Evelyn Eastmond. The scratch programming language and environment. *ACM Transactions on Computing Education*, 10(4):1–15, 2010. doi: 10.1145/1868358.1868363.
- Orni Meerbaum-Salant, Michal Armoni, and Mordechai Ben-Ari. Learning computer science concepts with scratch. *Computer Science Education*, 23(3):239–264, 2013. doi: 10.1080/08993408.2013.832022.
- Marcus Messer, Neil C. C. Brown, Michael Kölling, and Miaoqing Shi. Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education*, 24(1), February 2024. doi: 10.1145/3636515. URL <https://doi.org/10.1145/3636515>.
- Peter Naur. Automatic grading of students’ algol programming. *BIT*, 4(3):177–188, 1964. doi: 10.1007/BF01956028.
- Seymour Papert. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, New York, 1980.
- Seymour Papert and Idit Harel. Situating constructionism. In Idit Harel and Seymour Papert, editors, *Constructionism*, pages 1–11. Ablex Publishing, Norwood, NJ, 1991.
- Jean Piaget and Bärbel Inhelder. *The Psychology of the Child*. Basic Books, New York, 1969.
- Janet C. Read and Mathilde M. Bekker. The nature of child computer interaction. In *Proceedings of the 22nd British HCI Group Annual Conference on People and Computers*, pages 163–170. British Computer Society, 2008.
- Mitchel Resnick, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. Scratch: Programming for all. *Communications of the ACM*, 52(11):60–67, 2009. doi: 10.1145/1592761.1592779.

- José-Manuel Sáez-López, Marcos Román-González, and Esteban Vázquez-Cano. Visual programming languages integrated across the curriculum in elementary school: A two year case study using scratch in five schools. *Computers & Education*, 97:129–141, 2016. doi: 10.1016/j.compedu.2016.03.003.
- Sue Sentance, Jane Waite, Steve Hodges, Emily MacLeod, and Lucy Yeomans. Creating cool stuff: Pupils’ experience of the bbc micro:bit. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education*, pages 531–536, 2017a. doi: 10.1145/3017680.3017749.
- Sue Sentance, Jane Waite, Lucy Yeomans, and Emily MacLeod. Teaching with physical computing devices: The bbc micro:bit initiative. In *Proceedings of the 12th Workshop on Primary and Secondary Computing Education*, pages 87–96, 2017b. doi: 10.1145/3137065.3137083.
- Karim Shabani, Mohammad Khatib, and Saman Ebadi. Vygotsky’s zone of proximal development: Instructional implications and teachers’ professional development. *English Language Teaching*, 3(4):237–248, 2010. doi: 10.5539/elt.v3n4p237.
- Valerie J. Shute. Focus on formative feedback. *Review of Educational Research*, 78(1):153–189, 2008. doi: 10.3102/0034654307313795.
- John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988. doi: 10.1207/s15516709cog1202_4.
- L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, Cambridge, MA, 1980.
- David Weintrop and Uri Wilensky. To block or not to block, that is the question: Students’ perceptions of blocks-based programming. In *Proceedings of the 14th International Conference on Interaction Design and Children*, pages 199–208. ACM, 2015. doi: 10.1145/2771839.2771860.
- Jeannette M. Wing. Computational thinking. *Communications of the ACM*, 49(3):33–35, 2006. doi: 10.1145/1118178.1118215.